



OpsBrain

The operations brain for the plant floor.

Ask a plant's manuals, work orders and safety standards in plain language, and get a cited answer back in seconds — plus a compliance check and a map of how the equipment connects.

ET AI Hackathon 2.0 · Phase 2 — Build Sprint

Problem Statement 8 — Industrial Knowledge Intelligence

Divyanshi Jain · github.com/techwack/opsbrain

What OpsBrain is

OpsBrain is a copilot that sits on top of a plant's existing documents — OEM manuals, P&IDs, work-order history, OISD and Factory Act standards, incident reports — and lets an engineer ask it anything. It answers in plain language, cites the exact document and page it used, and tells you how confident it is. On top of that it does two things a search box can't: it audits an asset against safety standards and flags the gaps, and it draws the relationships between equipment, failure modes and the rules that govern them.

The problem I set out to solve

I kept coming back to one line from the brief: the information almost always exists, but nobody can find it in time. A single refinery or steel plant runs on tens of thousands of pages, written over decades, sitting in binders, shared drives and a couple of generations of software that don't talk to each other. When a pump trips on the night shift, the maintenance history, the OEM procedure and the relevant safety clause are all there somewhere. The engineer just can't get to them fast enough — so they rely on memory, or on the one person who was around for the last failure.

That has three costs, and they compound:

Time lost

Studies of process industries put roughly a third of an engineer's day into hunting for information that already exists.

Mistakes repeated

The last work order on an asset is rarely read before the next job, so the same failure comes back.

Risk discovered late

Compliance obligations live inside long PDFs that nobody re-reads until after an incident forces it.

None of this is a modelling problem. It's a retrieval problem wearing a domain coat — which is exactly what a grounded, cited LLM system is good at, if you build it carefully.

WHAT I BUILT

Three things, one corpus

Everything below runs off the same indexed set of documents. Upload more and all three features get smarter at once.

1 — Ask, with citations you can check

A technician types a question the way they'd say it out loud. OpsBrain retrieves the most relevant passages, hands them to Claude with strict instructions to stay grounded, and returns an answer where every claim is tagged back to a source. A confidence score (built from retrieval similarity) tells the reader when to trust it and when to go read the manual themselves. If anything implies a safety or compliance risk, the answer flags it.

The screenshot shows the OpsBrain interface. On the left is a sidebar with navigation options: 'Ask OpsBrain' (selected), 'Compliance Audit', 'Knowledge Graph', and 'Documents'. At the bottom of the sidebar, it shows '5 documents' and '47 indexed chunks', with a link to 'Start the backend for live data'. The main area is titled 'Ask OpsBrain' with the subtitle 'Grounded answers from your plant's manuals, work orders & standards — with citations.' A question box contains the text 'How do I replace the mechanical seal on P-204 safely?'. The answer is displayed in a box with a blue icon, stating 'To replace the mechanical seal on P-204 safely:' followed by five numbered steps. Each step includes specific equipment references and safety instructions, with some steps marked as critical (e.g., 'double block and bleed'). Below the answer is a 'confidence' bar at 91%. Underneath, a 'SOURCES' section lists three references: '[S1] 01_pump_P-204_OEM_manual.md' (relevance 0.93), '[S2] 02_workorder_history_unit2.md' (relevance 0.88), and '[S3] 03_OISD-STD-105_excerpt.md' (relevance 0.85). At the bottom, there is a text input field with the placeholder 'Ask about equipment, failures, procedures, standards...' and a blue submit button with an upward arrow.

OpsBrain
Industrial Knowledge Intelligence

Ask OpsBrain

Compliance Audit

Knowledge Graph

Documents

5 documents

47 indexed chunks

Start the backend for live data

Ask OpsBrain
Grounded answers from your plant's manuals, work orders & standards — with citations.

How do I replace the mechanical seal on P-204 safely?

To replace the mechanical seal on P-204 safely:

1. Raise a valid **Permit-to-Work** — mandatory for seal jobs per OISD-STD-105 Clause 4.2 [S3].
2. Isolate suction valve **V-204A** and discharge valve **V-204B**, de-energize motor **M-204** at MCC-2, and apply **LOTO** [S1].
3. **V-204B** has a history of passing (WO-2024-0512) — confirm positive isolation; use **double block and bleed** if integrity is in doubt [S1].
4. Replace the **John Crane Type 2100** seal faces and O-rings **as a set**; torque gland nuts to **25 N·m** in a star pattern [S1].
5. Verify the **API Plan 11+62** flush/quench is clear before restart [S2].

confidence 91%

SOURCES

- [S1] 01_pump_P-204_OEM_manual.md · p.1 · relevance 0.93
Before any intervention on P-204, isolate suction valve V-204A and discharge valve V-204B, de-energize motor M-204... apply LOTO. A valid Permit-to-Work is mandatory.
- [S2] 02_workorder_history_unit2.md · p.1 · relevance 0.88
Flush plan upgraded to API Plan 11 + 62 (quench). Root cause: inadequate flush margin under high suction temperature.
- [S3] 03_OISD-STD-105_excerpt.md · p.1 · relevance 0.85
A valid written Permit-to-Work shall be issued before any maintenance on hydrocarbon service equipment.

Ask about equipment, failures, procedures, standards...

Asking how to replace the seal on pump P-204. The steps, the torque spec and a safety flag — each line cited back to the OEM manual, work-order history or the OISD standard.

2 — A compliance check that runs before the auditor does

Point it at an asset and OpsBrain pulls the operational record and the relevant standard, then asks the model to find concrete deviations — not vague advice, but specific gaps tied to a clause and ranked by severity, each with a recommended action.

OpsBrain
Industrial Knowledge Intelligence

Ask OpsBrain

Compliance Audit

Knowledge Graph

Documents

5 documents

47 indexed chunks

Start the backend for live data

Compliance Audit

Auto-audit equipment against OISD & Factory Act standards for ranked gaps.

P-204 mechanical seal

Run audit

2 gaps found for 'P-204 mechanical seal' — one high-severity RCFA omission and one isolation-integrity gap.

HIGH

No RCFA after repeat seal failures

OISD-STD-105 Clause 5.1 · S1

P-204 logged three seal failures across 2023–24, including a high-potential near-miss (NM-2023-058), but no documented Root Cause Failure Analysis was raised. The standard mandates an RCFA after more than two seal failures in any 12-month period.

→ Raise and track a formal RCFA on the P-204 seal failures to closure; review flush adequacy at peak suction temperature.

MEDIUM

Isolation integrity not verified

OISD-STD-105 Clause 6.3 · S2

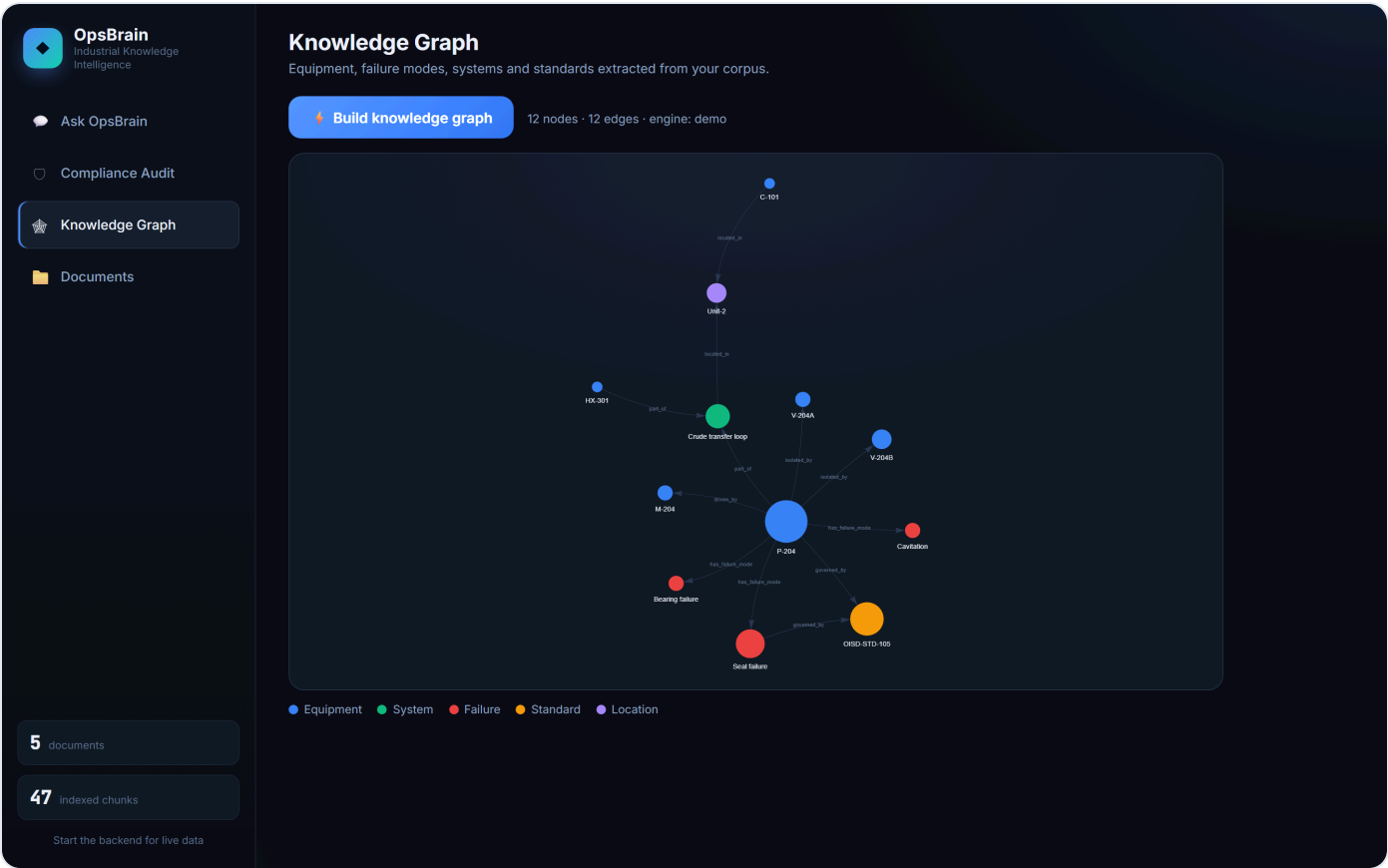
Discharge valve V-204B has a recorded passing history (WO-2024-0512) yet is still used as a primary isolation point for seal work.

→ Mandate double block and bleed (or a spade) and a positive-isolation check on the P-204 PTW until V-204B integrity is re-validated.

Auditing P-204's seal history surfaces a **HIGH** missing root-cause analysis after repeat failures, and a **MEDIUM** isolation gap on a valve with a passing history — both tied to OISD-STD-105 clauses.

3 — The plant as a knowledge graph

From the same documents, OpsBrain extracts equipment tags, failure modes, systems, standards and locations, and draws the edges between them. It's how you see that one pump sits at the centre of three failure modes and a safety standard — at a glance.

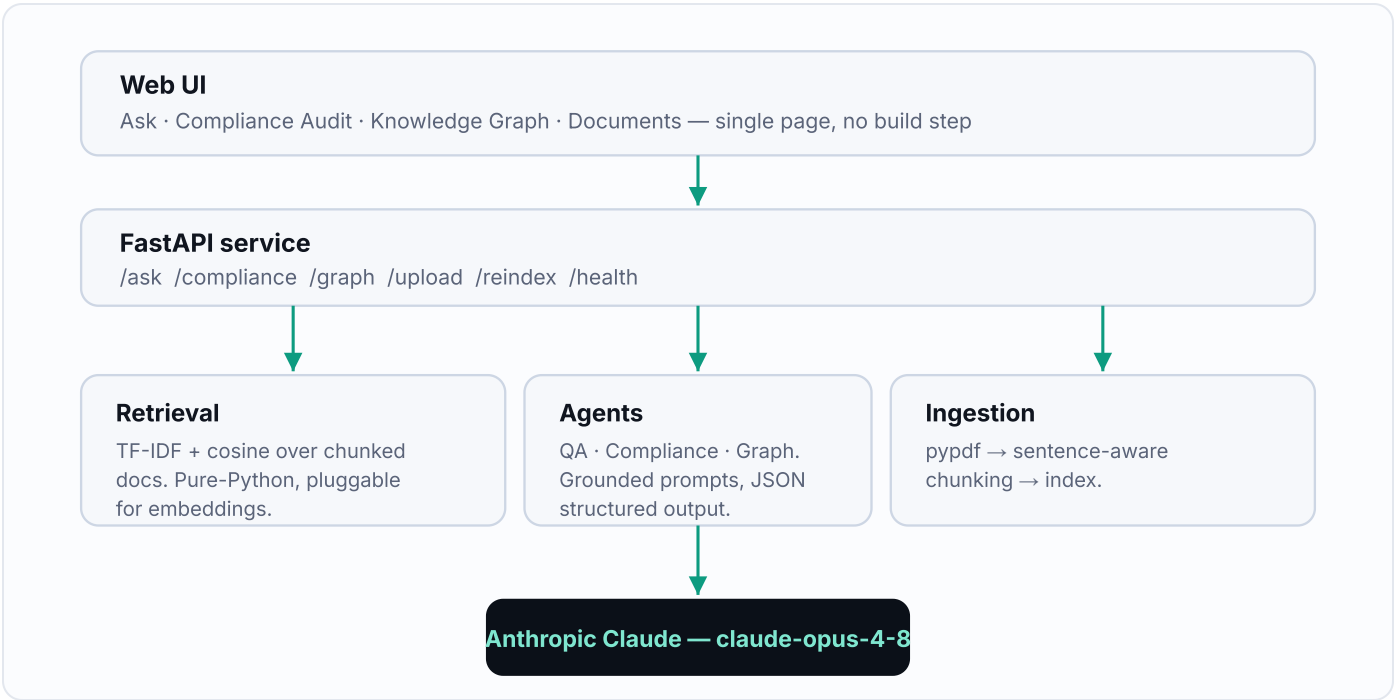


Equipment (blue), failure modes (red), systems (green) and the standards that govern them (amber), extracted automatically and laid out by relationship.

HOW IT WORKS

The architecture

One service serves the API and the UI. Retrieval runs with no external dependencies; Claude does the reasoning. Nothing here needs a GPU or a cloud account to run.



Retrieval and the knowledge graph run offline; the agents call Claude when a key is present. With no key, answers fall back to cited extracts — the app is never a blank screen.

Tech stack

Backend	Python, FastAPI, Uvicorn — one process serves both the API and the UI
Retrieval	scikit-learn TF-IDF + cosine similarity, behind a vector-store interface so it can be swapped for embeddings
Reasoning	Anthropic Claude (claude-opus-4-8) for answers, compliance analysis and graph extraction
Ingestion	pypdf, with sentence-aware overlapping chunking
Knowledge graph	NetworkX for centrality, vis-network for the interactive render
Front end	A single-page app with no build step — plain HTML/CSS/JS, served by FastAPI

The choices I'd defend in a review

A hackathon rewards finishing, but it's the trade-offs that show judgement. Here are the four I made deliberately.

Single service instead of a microservice split.

FastAPI serves the API and the front end from one process, so the whole thing runs with one command. For a solo build and a live demo, that reliability beats premature separation every time.

TF-IDF retrieval instead of a heavyweight vector database.

A managed vector store wanted either a GPU or a C++ build chain just to install — a bad bet on an unknown machine. TF-IDF clones and runs anywhere in seconds. Because it's behind a small interface, moving to embeddings later is a one-file change, not a rewrite.

It works with no API key.

Retrieval and the graph run offline, and answers degrade gracefully to cited extracts. A judge who clones the repo without a key still sees a working product.

Citations and a confidence score on every answer.

An operations tool that can't show its source is a liability, not an asset. Grounding isn't a feature here — it's the whole point.

Why it's worth it — the business case

The value is boring in the best way: give engineers their time back, stop repeating avoidable failures, and catch compliance gaps while they're still cheap to fix.

Heavy industry — steel, oil & gas, pharma, power — is the backbone of Indian manufacturing, and it's exactly the sector drowning in this kind of documentation. A few percentage points of engineer time, across a plant, is crores a year. One avoided unplanned shutdown pays for the system many times over.

And it scales sideways with almost no rework: the engine is indifferent to which plant it serves. Change the corpus, not the code.

~35%

of engineer time spent searching is the pool OpsBrain pulls from.

1 cmd

to run the whole stack — API, UI, retrieval and sample corpus.

4

industries it ports to with no code change: steel, oil & gas, pharma, power.

RUNNING IT

From clone to running in under a minute

```
cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt

# optional - turns on Claude-synthesised answers
copy ..\.env.example .env      # add ANTHROPIC_API_KEY

uvicorn app.main:app --reload  # open http://localhost:8000
```

The sample plant corpus — a Unit-2 crude-transfer loop with pump P-204, its OEM manual, two years of work orders, an OISD standard excerpt, a near-miss report and a P&ID description — is indexed automatically on first boot, so there's something to ask immediately. Drop your own PDFs in through the Documents tab and they're chunked and indexed on upload.

WHERE IT GOES NEXT

Same engine, more inputs

Now	Cited Q&A, compliance gap detection, knowledge graph and document upload — working end to end.
Next	OCR for scanned P&IDs, so legacy drawings become searchable instead of opaque images.
Then	Connectors into CMMS / SCADA (SAP PM, Maximo) so live work orders and sensor readings flow in.
Later	Predictive maintenance scoring built on the work-order and sensor history the system already holds.

The honest summary: the hard, demoable core is done and runs anywhere. What's left is integration and depth — and the architecture was chosen so that adding those doesn't mean starting over.